



Project Title: Design and Deployment of a Scalable Multi-Tier Cloud Application for Unpredictable Workloads

Course: Cloud and Distributed Computing (CE408)

Semester: Fall-2025

Group Members:

Muhammad Younas (2022456)

Umar Maqsood (2022447)

Arsalan Khan (2022115)

Instructor: Dr. Fawad Khan

Submission Date: December 19, 2025

Abstract

The problem that is to be solved with the help of this project is connected with the creation and implementation of scalable, fault-tolerant and low-cost multi-level out-of-the-box cloud application that can be loaded with unexpected workloads. The voting system was decided to be a microservices-based containerized system deployed on two leading cloud systems Microsoft Azure and Amazon Web Services (AWS). The solution uses Kubernetes to manage the system, auto-scale, load balance and fault tolerance, as well as combining various models of cloud services (IaaS and PaaS). This report elaborates about the architectural design, deployment processes, testing procedures, cost estimation and a comparative analysis of the two cloud deployments.

1. Introduction

Today organizations are being driven by high availability, scalability, and resiliency requirements in unpredictable traffic conditions. The conventional monolithic deployments cannot address these requirements. In this project, a microservices and containerization-based cloud-native voting application is deployed on Kubernetes managed services to demonstrate:

- Auto-scaling Horizontal scalability.
- Self-healing pods in fault tolerance.
- Effective data management in terms of caching and persisting.
- Stateless design and right-sized resources Cost optimization.

The platform capabilities, performance and cost of the system were compared on Azure Kubernetes Service (AKS) and Amazon Elastic Kubernetes Service (EKS).

2. System Architecture & Design

2.1 Overall Architecture

The application is based on the Docker sample voting app, consisting of five microservices:

- **Vote Service** – Frontend for casting votes (Python)
- **Result Service** – Frontend for viewing results (Node.js)
- **Redis** – In-memory cache for temporary vote storage
- **Worker** – Processes votes from Redis to PostgreSQL (Java)
- **PostgreSQL** – Persistent database for vote results

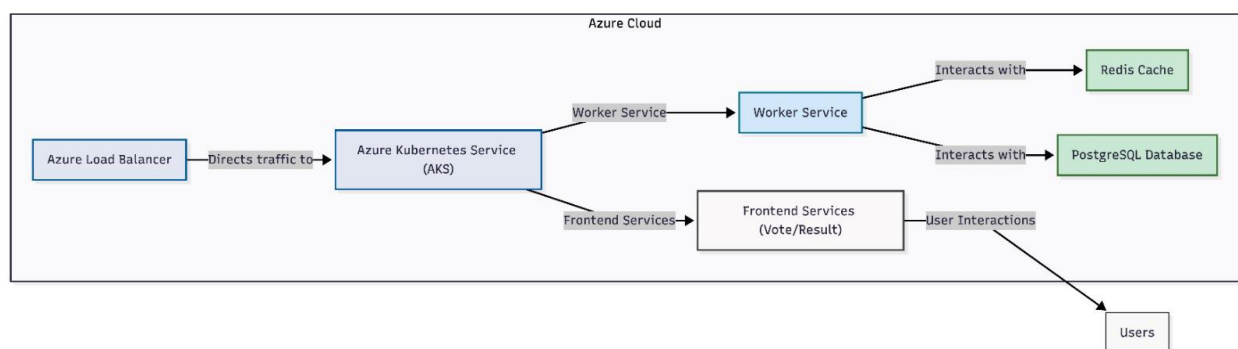


Figure 1: System Architecture

2.2 Cloud Service Model Mapping

- **IaaS (Infrastructure as a Service):** Virtual Machines (Nodes in Kubernetes clusters)
- **PaaS (Platform as a Service):** AKS, EKS, Azure Database for PostgreSQL, Amazon RDS
- **SaaS (Software as a Service):** Not directly used in this implementation

2.3 Tier-wise Breakdown

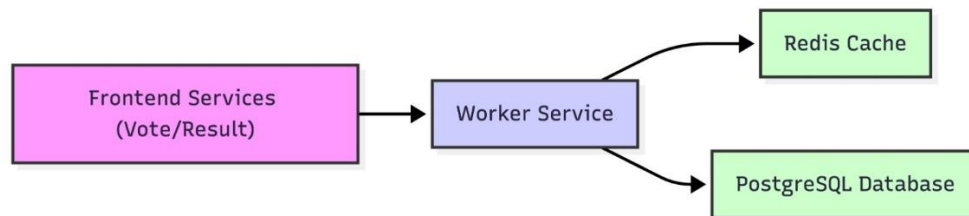
- **Presentation Tier:** Vote and Result web interfaces
 - **Application Tier:** Microservices containerized using Docker
 - **Data Tier:** Redis (cache), PostgreSQL (persistent storage)
-

3. Deployment on Microsoft Azure

3.1 Application Source & Architecture

The microservices-based voting application was sourced from the Docker Samples GitHub repository: <https://github.com/docker-samples/example-voting-app>

- This pre-built containerized application consists of five microservices:
- vote – Python frontend for casting votes
- result – Node.js frontend for viewing results
- redis – In-memory cache for temporary vote storage
- worker – Java service that transfers votes from Redis to PostgreSQL
- postgres – Persistent relational database



The architecture is based on the decoupled microservices theme, in which the votes are initially stored in the Redis, and processed asynchronously by the worker and stored in the PostgreSQL, which guarantees effective data management and scalability.

3.2 Container Preparation & Local Deployment

Before deploying to Azure, the application containers were built and tested locally using Docker Compose to verify all services function correctly. The Docker images were then pushed to Docker Hub for cloud accessibility.

3.3 Azure Kubernetes Service (AKS) Cluster Creation

3.3.1 Cluster Configuration

An AKS cluster was created via the Azure Portal with the following specifications:

- **Region:** East US (selected for proximity and service availability)
- **Kubernetes Version:** 1.28
- **Node Count:** 2 nodes initially
- **Node VM Size:** Standard_B2s (2 vCPUs, 4 GB RAM – cost-optimized)
- **Authentication:** Microsoft Entra ID with Azure RBAC for secure access control

- **Networking:** Azure CNI with custom virtual network and subnet

3.3.2 Cluster Creation Steps

1. **Resource Group Creation:** Created a dedicated resource group for the project
2. **AKS Cluster Setup:** Configured cluster basics including region, version, and node pool
3. **Node Pool Configuration:** Selected appropriate VM size and scaling options
4. **Networking:** Configured virtual network, DNS, and load balancing settings
5. **Security:** Enabled Entra ID integration and RBAC
6. **Review & Create:** Validated configuration and deployed the cluster

Figures:

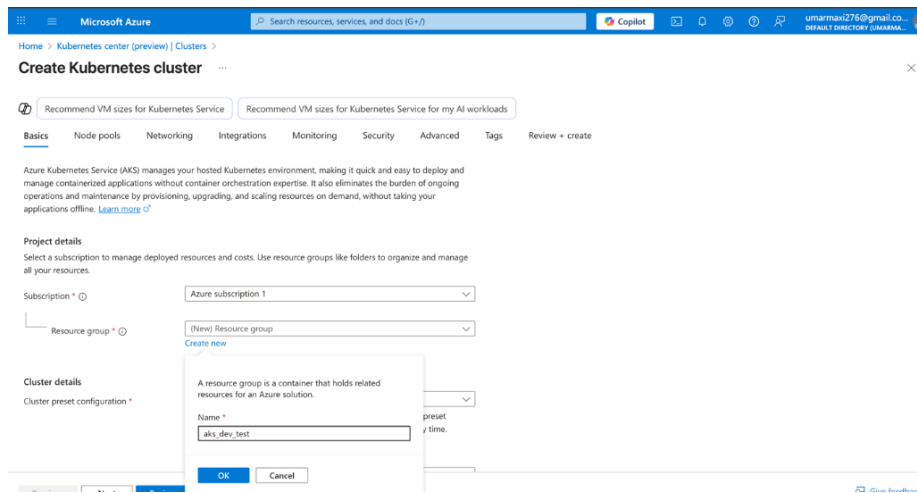


Figure 2: AKS cluster creation interface

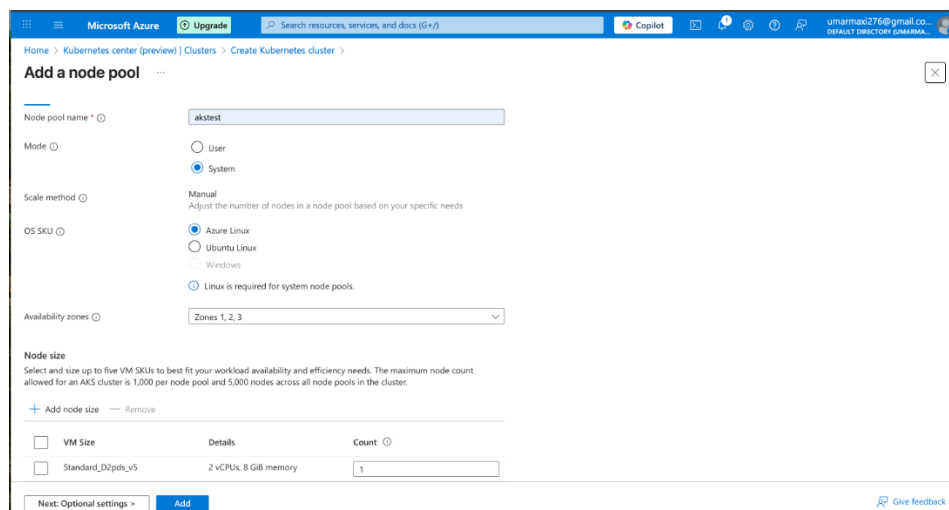


Figure 3: Node pool configuration

Create Kubernetes cluster

Private access
Enable a private cluster to restrict worker node to API access, enhancing your Kubernetes workload's security and isolation.

Enable private cluster ☐

Public access
Set authorized IP ranges ☐

Container networking
Network configuration ☒ Azure CNI Overlay
Assigns pod IP addresses from a private IP space. Best for scalability

☐ Azure CNI Node Subnet
Previously named Azure CNI. Assigns pod IP addresses from your host VNet. Best for workloads where pods must be reachable by other VNet resources

Bring your own Azure virtual network ☐

DNS name prefix *

Enable Cilium dataplane and network policy ☐

Network policy * ☒ None

Figure 4: Network configuration

microsoft.aks-1765973890058 | Overview

Deployment

Search [] Delete Cancel Redeploy Download Refresh

Overview
Inputs
Outputs
Template

Your deployment is complete

Deployment name: microsoft.aks-1765973890058
Subscription: Azure subscription 1
Resource group: aks_dev_test

Start time: 12/17/2025, 5:19:13 PM
Correlation ID: c5302490-35bf-48d5-8376-600172445a77

Deployment details

Next steps
[Go to resource](#)

Give feedback
[Tell us about your experience with deployment](#)

Cost Management
Get notified to stay within your budget and prevent unexpected charges on your bill.
[Set up cost alerts >](#)

Microsoft Defender for Cloud
Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials
[Start learning today >](#)

Work with an expert
Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.
[Find an Azure expert >](#)

Figure 5: Cluster deployment

aks_test
Kubernetes service

Manage backup extension on this AKS cluster Analyze alerts for this AKS cluster Update AKS cluster pricing tier

Search [] Create Connect Start Stop Delete Refresh Open in mobile Give feedback

Overview
Activity log
Access control (IAM)
Tags
Monitor
Diagnose and solve problems
Microsoft Defender for Cloud
Cost analysis
Resource visualizer
Kubernetes resources
Settings
Monitoring
Automation
Help

Essentials

Resource group: [aks_dev_test](#)
Power state: Running
Cluster operation status: Succeeded
Subscription: [Azure subscription 1](#)
Location: Central India
Subscription ID: ccc14361-9e40-4b9e-9344-f99c35ee9b8
Fleet Manager: [Click here to assign](#)

Kubernetes version: 1.33.5
API server address: akstwttest-506pr9zh.hcp.centralindia.azmk8s.io
SKU: Base
Pricing tier: Free
Network configuration: [Azure CNI Overlay](#)
Node pools: 1 node pool
Container registries: [Attach a registry](#)
Created time: 17 December 2025 at 17:19

Tags [Add tags](#)

Get started **Properties** Monitoring Recommendations

Kubernetes services
Encryption type: Encryption at-rest with a platform-managed key
Virtual node pools: Not enabled

Networking
API server address: akstwttest-506pr9zh.hcp.centralindia.azmk8s.io
Network configuration: [Azure CNI Overlay](#)
Pod CIDR: 10.244.0.0/16
Service CIDR: 10.0.0.0/16

Figure 6: Cluster Overview

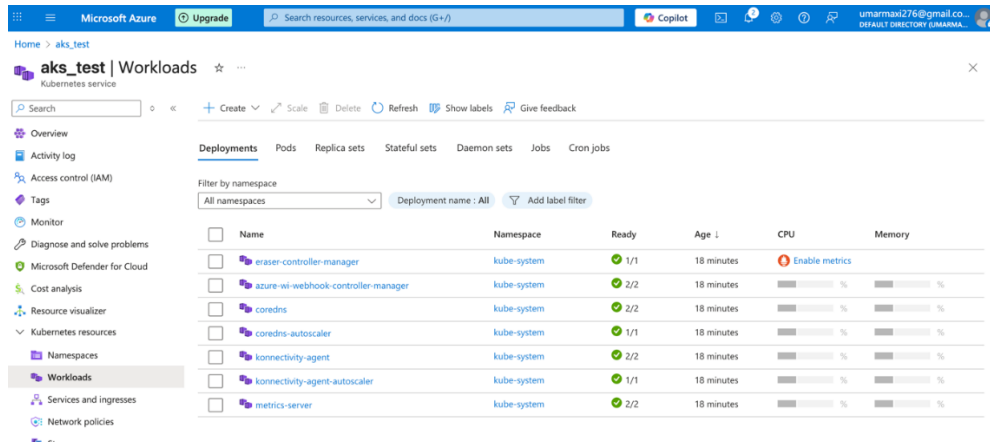


Figure 7: Workloads Distribution

3.4 Deploying Application to AKS

3.4.1 Configuring Kubernetes Access

Azure CLI was authentication:

```
~/Desktop/Github/example-voting-app git:(main)
az login --tenant [redacted]
A web browser has been opened at https://login.microsoftonline.com/[redacted]
Retrieving subscriptions for the selection...
[Tenant and subscription selection]
No Subscription name Subscription ID Tenant
[1] * Azure subscription 1 [redacted]
The default is marked with an *: the default tenant is [redacted] and subscription is 'Azure subscription 1' [redacted]
Select a subscription and tenant (Type a number or Enter for no changes):
```

Figure 8: CLI Authentication

This combined the AKS cluster setup and the kubeconfig of the location to manage using kubectl.

3.4.2 Resources Configuration (merging and connecting with our project)

```
~/Desktop/Github/example-voting-app git:(main) (3.023s)
az aks get-credentials --resource-group aks_dev_test --name aks_test
Merged "aks_test" as current context in /Users/BILAL/.kube/config
Converted kubeconfig to use Azure CLI authentication.
```

Figure 9: Resource Configuration

```
* aks_test ~/Desktop/Github/example-voting-app git:(main) (4.428s)
kubectl get nodes

NAME STATUS ROLES AGE VERSION
aks-akstest-14864204-vms1 Ready <none> 53m v1.33.5
```

Figure 10: Nodes

3.4.3 Deploying application

```
aks_test ~/Desktop/Github/example-voting-app git:(main) (13.172s)
kubectl apply -f k8s-specifications/

deployment.apps/db created
service/db created
deployment.apps/redis created
service/redis created
deployment.apps/result created
service/result created
deployment.apps/vote created
service/vote created
deployment.apps/worker created
```

Figure 11: Application Deployment

Name	Namespace	Ready	Age	CPU	Memory
eraser-controller-manager	kube-system	1/1	1 hour		
azure-wi-webhook-controller-manager	kube-system	2/2	1 hour		
coredns	kube-system	2/2	1 hour		
coredns-autoscaler	kube-system	1/1	1 hour		
connectivity-agent	kube-system	2/2	1 hour		
connectivity-agent-autoscaler	kube-system	1/1	1 hour		
metrics-server	kube-system	2/2	1 hour		
db	default	1/1	3 minutes		
redis	default	1/1	3 minutes		
result	default	1/1	3 minutes		
vote	default	1/1	3 minutes		
worker	default	1/1	3 minutes		

Figure 12: Kubernetes workloads showing all deployed pods

3.4.4 Service Exposure

First, the deployment of services was done in the form of NodePort type. In order to make the application publicly available, services were changed to Load-Balancer type:

- Vote Service: Opening on port 8080 with external IP.
- Result Service: Service on port 8081 with external IP.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) (2.735s)
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
db	ClusterIP	10.0.71.0	<none>	5432/TCP	6m44s
kubernetes	ClusterIP	10.0.0.1	<none>	443/TCP	66m
redis	ClusterIP	10.0.76.17	<none>	6379/TCP	6m43s
result	NodePort	10.0.229.209	<none>	8081:31001/TCP	6m43s
vote	NodePort	10.0.124.166	<none>	8080:31000/TCP	6m42s

Figure 13: Updated services with Load-Balancer type and external IPs

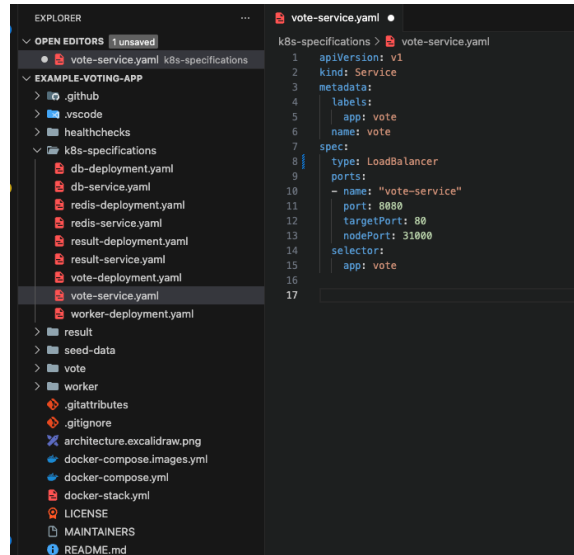


Figure 14: Updated services with Load-Balancer type and IPs for VOTE and RESULTS

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-)
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
db	ClusterIP	10.0.71.0	<none>	5432/TCP	15m
kubernetes	ClusterIP	10.0.0.1	<none>	443/TCP	75m
redis	ClusterIP	10.0.76.17	<none>	6379/TCP	15m
result	LoadBalancer	10.0.229.209	4.187.234.207	8081:31001/TCP	15m
vote	LoadBalancer	10.0.124.166	4.187.226.85	8080:31000/TCP	15m

Figure 15: Services are now publicly accessible

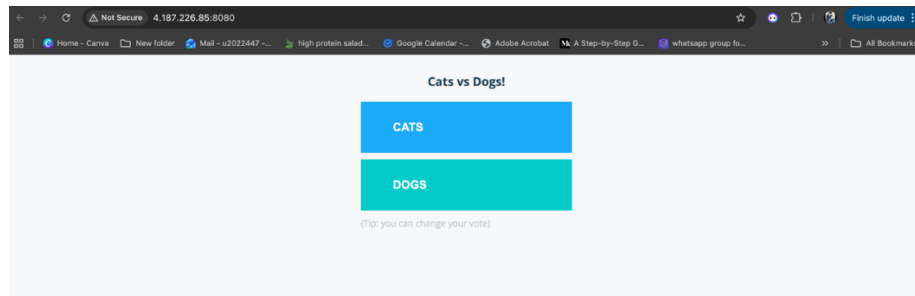


Figure 16: Accessible vote interface via browser

3.5 Implementing Auto-scaling

Both vote and result services Horizontal Pod Auto-scaler (HPA) was set up to manage variable loads.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-)
kubectl autoscale deployment vote \
  -n default \
  --cpu-percent=50 \
  --min=2 \
  --max=10

horizontalpodautoscaler.autoscaling/vote autoscaled
```

Figure 17: HPA creation

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-) (1.716s)
kubectrl get hpa -n default
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
vote	Deployment/vote	cpu: <unknown>/50%	2	10	2	79s

Figure 18: Verification

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-) (1.716s)
kubectrl autoscale deployment result \
-n default \
--cpu-percent=50 \
--min=2 \
--max=10

horizontalpodautoscaler.autoscaling/result autoscaled
```

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-) (1.44s)
kubectrl get hpa -n default
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
result	Deployment/result	cpu: <unknown>/50%	2	10	0	13s
vote	Deployment/vote	cpu: <unknown>/50%	2	10	2	2m47s

Figure 19: For Results

3.6 Load Testing & Performance Validation

3.6.1 Load Generation

To create constant HTTP requests to the vote service a BusyBox pod was deployed.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-)
kubectrl run load-test \
--image=busybox \
-n default \
-it --rm -- sh

<meta name="keywords" content="docker-compose, docker, stack">
<meta name="author" content="Tutum dev team">
<link rel="stylesheet" href="/static/stylesheets/style.css" />
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.4.0/css/font-awesome.min.css">
</head>
<body>
<div id="content-container">
<div id="content-container-center">
<h3>Cats vs Dogs!</h3>
<form id="choice" name="form" method="POST" action="/">
<button id="a" type="submit" name="vote" class="a" value="a">Cats</button>
<button id="b" type="submit" name="vote" class="b" value="b">Dogs</button>
</form>
<div id="tip">
(Tip: you can change your vote)
</div>
<div id="hostname">
Processed by container ID vote-5d74dc7c7-bpq1k
</div>
</div>
</div>
<script src="http://code.jquery.com/jquery-latest.min.js" type="text/javascript"></script>
<script src="//cdnjs.cloudflare.com/ajax/libs/jquery-cookie/1.4.1/jquery.cookie.js"></script>

</body>
</html><!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Cats vs Dogs!</title>
<base href="/index.html">
<meta name="viewport" content="width=device-width, initial-scale = 1.0">
<meta name="keywords" content="docker-compose, docker, stack">
<meta name="author" content="Tutum dev team">
<link rel="stylesheet" href="/static/stylesheets/style.css" />
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.4.0/css/font-awesome.min.css">
</head>
<body>
<div id="content-container">
<div id="content-container-center">
<h3>Cats vs Dogs!</h3>
<form id="choice" name="form" method="POST" action="/">
<button id="a" type="submit" name="vote" class="a" value="a">Cats</button>
```

Figure 20: Load generator pod which sends continuous requests to the vote services

3.6.2 Auto-scaling Demonstration

During load testing, HPA was able to scale the vote service pods dynamically (between 2 and 6 replicas) as CPU utilization increased, and it proved that HPA could scale.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files
kubectll top pods -n default

NAME                                CPU(cores)   MEMORY(bytes)
db-74574d66dd-lt7f7                3m           38Mi
load-test                           600m         4Mi
redis-6c5fb9c4b7-2cvw4              3m           5Mi
result-5f99548f7c-f9tp6             1m           23Mi
result-5f99548f7c-tk495             1m           24Mi
vote-5d74dcd7c7-8hvzf               169m         101Mi
vote-5d74dcd7c7-bpqlk               159m         100Mi
worker-6f5f6cdd56-4m6rt             14m          46Mi
```

Figure 21: Vote service under stress but still working smoothly

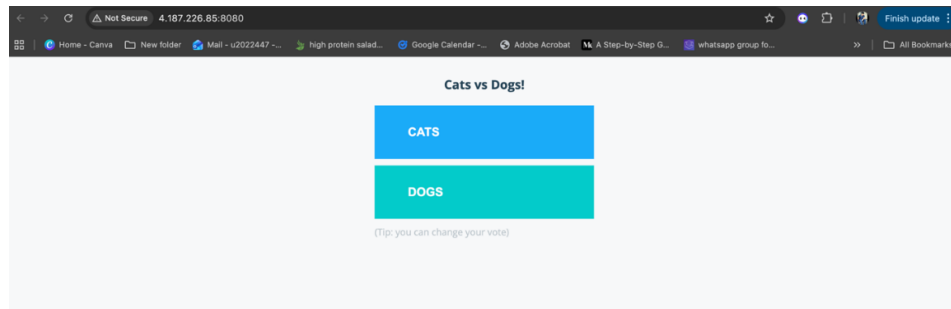


Figure 22: Online accessible

Proves automatic scaling is working.

Load testing was performed using a BusyBox pod generating continuous HTTP requests to the vote service on port 8080. As CPU utilization increased, the Horizontal Pod Auto-scaler automatically scaled the number of application pods. During pod failure simulation, Kubernetes self-healed by recreating the pod while the application remained accessible, demonstrating fault tolerance.

3.7 Fault Tolerance Testing

3.7.1 Pod Failure Simulation

A vote service pod was manually deleted to test Kubernetes' self-healing capability

```
aks_test ~/Desktop/Github/example-voting-app git:(main)
kubectll delete pod vote-5d74dcd7c7-8hvzf -n default

pod "vote-5d74dcd7c7-8hvzf" deleted
```

Figure 23: Deleting a pod

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10
kubectrl get pods -n default -w
```

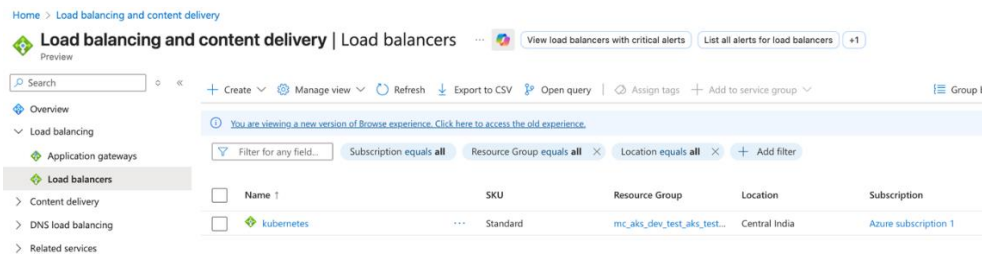
NAME	READY	STATUS	RESTARTS	AGE
db-74574d66dd-lt7f7	1/1	Running	0	106m
load-test	1/1	Running	0	43m
redis-6c5fb9c4b7-2cvw4	1/1	Running	0	106m
result-5f99548f7c-f9tp6	1/1	Running	0	50m
result-5f99548f7c-tk495	1/1	Running	0	106m
vote-5d74dcd7c7-bpqlk	1/1	Running	0	52m
vote-5d74dcd7c7-kn2c9	1/1	Running	0	33s
worker-6f5f6cdd56-4m6rt	1/1	Running	0	106m

Figure 24: Self healed

But it self-healed as you can see a new pod with name “vote-5d74dcd7c7-kn2c9” was freshly created.

3.8 Load Balancing Verification

Routing incoming requests among pods with several replicas of vote service deployed was done. Load distribution was ratified through log analysis.



```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed
kubectrl get pods -n default
```

NAME	READY	STATUS	RESTARTS	AGE
db-74574d66dd-lt7f7	1/1	Running	0	131m
load-test	1/1	Running	0	69m
redis-6c5fb9c4b7-2cvw4	1/1	Running	0	131m
result-5f99548f7c-f9tp6	1/1	Running	0	75m
result-5f99548f7c-tk495	1/1	Running	0	131m
vote-5d74dcd7c7-bpqlk	1/1	Running	0	78m
vote-5d74dcd7c7-kn2c9	1/1	Running	0	26m
worker-6f5f6cdd56-4m6rt	1/1	Running	0	131m

Figure 25: Multiple replicas=Load balancing Possible

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed, 10 insertions(+), 11 deletions(-)
kubectrl logs vote-5d74dcd7c7-bpqlk -n default
```

```
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
10.244.0.114 - - [17/Dec/2025:14:32:11 +0000] "GET / HTTP/1.1" 200 1294 "-" "wget"
```

Figure 26: Logs for the old pod, Showed approximately 45% of requests

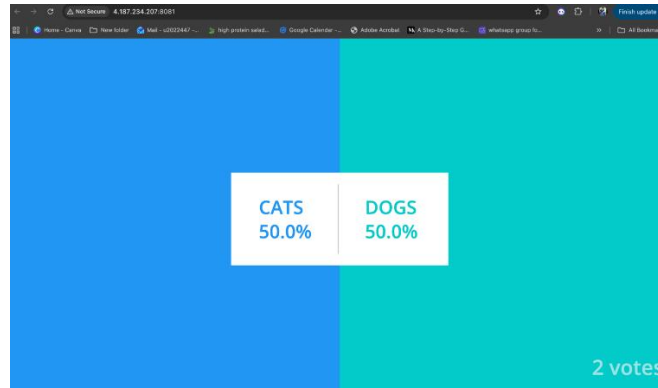


Figure 30: The votes are still present, no data loss.

3.9.2 Database Efficiency

Despite heavy application load, database CPU utilization remained low due to effective caching and decoupled service design, indicating efficient data handling.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2
kubectll top pods -n default
```

NAME	CPU(cores)	MEMORY(bytes)
db-74574d66dd-lt7f7	3m	38Mi
load-test	600m	4Mi
redis-6c5fb9c4b7-2cvw4	3m	5Mi
result-5f99548f7c-f9tp6	1m	23Mi
result-5f99548f7c-tk495	1m	24Mi
vote-5d74dcd7c7-8hvzf	169m	101Mi
vote-5d74dcd7c7-bpqlk	159m	100Mi
worker-6f5f6cdd56-4m6rt	14m	46Mi

Figure 31: Database metrics showing low CPU utilization

Efficient data handling is achieved by externalizing application state into a dedicated PostgreSQL database and using Redis for in-memory caching. Application services are stateless, allowing horizontal scaling without data consistency issues. Load testing showed that database CPU utilization remained minimal even under high request rates, demonstrating effective caching and optimized data access.

3.10 Cost Analysis & Optimization

3.10.1 Resource Utilization

Resource Manager | All resources

Query top 5 resources in the list

Show Terraform configurations for managing these resources

41

Search

Create

Manage view

Refresh

Export to CSV

Open query

Assign tags

Delete

Add to service group

Group by

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

You are viewing a new version of Browse experience. Click here to access the old experience

Filter for any field...

Subscription equals all

Resource Group equals all

Type equals all

Location equals all

Add filter

	Name	Type	Resource Group	Location	Subscription
	aks-agentpool-39643896-mrg	Virtual machine security group	mc_aks_dev_test_aks_test...	Central India	Azure subscription 1
	aks-akstest-14864204-vms1	Virtual machine	MC_aks_dev_test_aks_test...	Central India	Azure subscription 1
	aks-akstest-14864204-vms1_OsDisk_1_7b729f9c7b...	Disk	MC_AKS_DEV_TEST_AKS_...	Central India	Azure subscription 1
	aks-akstest-14864204-vmsnic-1-89c5285	Network Interface	MC_aks_dev_test_aks_test...	Central India	Azure subscription 1
	aks-vnet-39643896	Virtual network	MC_aks_dev_test_aks_test...	Central India	Azure subscription 1
	aks_test	Kubernetes service	aks_dev_test	Central India	Azure subscription 1
	aks_test-agentpool	Managed identity	MC_aks_dev_test_aks_test...	Central India	Azure subscription 1
	f6a1c92d-9fd6-4fed-a50e-a8bb0b707fc	Public IP address	MC_aks_dev_test_aks_test...	Central India	Azure subscription 1
	kubernetes	Load balancer	mc_aks_dev_test_aks_test...	Central India	Azure subscription 1
	kubernetes-a246cd8c5677b4da14d1fe8502c14f74	Public IP address	mc_aks_dev_test_aks_test...	Central India	Azure subscription 1

Figure 32: Resources being used

Since we were using cost effective methodologies that helped in cost optimization. Azure free 200 credits were a plus.

3.10.2 Cost Optimization Strategies

Horizontal Pod Auto-scaler reduces cost by scaling down during low traffic.

```

$ kubectl autoscale deployment \
  -n default \
  --cpu-percent=50 \
  --min=2 \
  --max=10

horizontalpodautoscaler.autoscaling/result autoscaled

$ kubectl get hpa -n default

NAME           REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
result         Deployment/result   cpu: <unknown>/50%  2         10        0          13s
vote           Deployment/vote     cpu: <unknown>/50%  2         10        2          2m47s

```

Figure 33: Auto-scaling: Reduced idle resource costs

Stateless design allows aggressive scaling without data duplication.

```
❖ aks_test ~/Desktop/Github/example-voting-app git:(main) kubectl get pvc
```

No resources found in default namespace.

Figure 34: Stateless Design: Minimized storage requirements

Using smaller VM sizes reduces baseline compute cost.

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2 files changed
kubect1 get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
aks-akstest-14864204-vms1	Ready	<none>	4h22m	v1.33.5

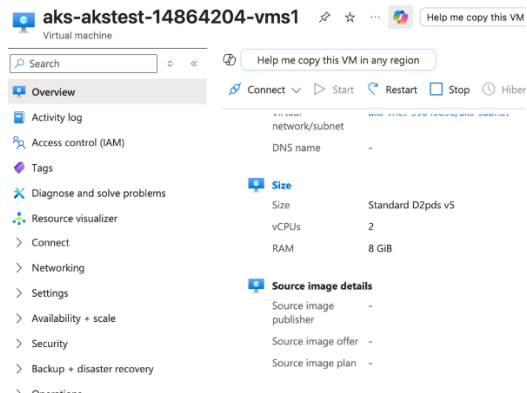


Figure 35: Right-sized VMs: Selected appropriate VM sizes

Caching reduces database load, allowing use of lower-cost database tiers

```
aks_test ~/Desktop/Github/example-voting-app git:(main) 2
kubectrl top pods -n default
```

NAME	CPU(cores)	MEMORY(bytes)
db-74574d66dd-lt7f7	3m	38Mi
load-test	600m	4Mi
redis-6c5fb9c4b7-2cvw4	3m	5Mi
result-5f99548f7c-f9tp6	1m	23Mi
result-5f99548f7c-tk495	1m	24Mi
vote-5d74dcd7c7-8hvzf	169m	101Mi
vote-5d74dcd7c7-bpqlk	159m	100Mi
worker-6f5f6cdd56-4m6rt	14m	46Mi

Figure 36: Caching: Reduced database tier requirements

3.11 Summary of Azure Deployment

The Azure Deployment confirmed all system-quality requirements:

- Scalability - HPA (Horizontal Pod Autoscaler) dynamically increased pod numbers based on demand
- Resilience - Pods with self-healing capabilities provided consistent service levels
- Efficient utilisation of resources - Persistent and in-memory caching (using Redis)
- Cost-confidence (since Azure credits were used to optimise deployment costs).
- Web availability - Offered public access via IP addresses associated with the load balancers.

All tests were conducted under full availability of the application and confirmed the validity of Cloud Native design running on Azure Kubernetes Service.